

ИЗУЧЕНИЕ ЭЛЕМЕНТОВ ШИФРОВАНИЯ И КОНТЕЙНИРИЗАЦИИ

Цель лабораторной работы:

Изучить особенности процесса создания и работы с Docker и шифрование файла с использованием GPG.

Задачи:

1. Генерация ключей
2. Шифрование файла
3. Расшифровка файла
4. Установка Docker
5. Запуск контейнера
6. Запуск собственного приложения в контейнере
- 7.

Используемое программное обеспечение:

Для выполнения лабораторной работы установленный дистрибутив ОС Linux.

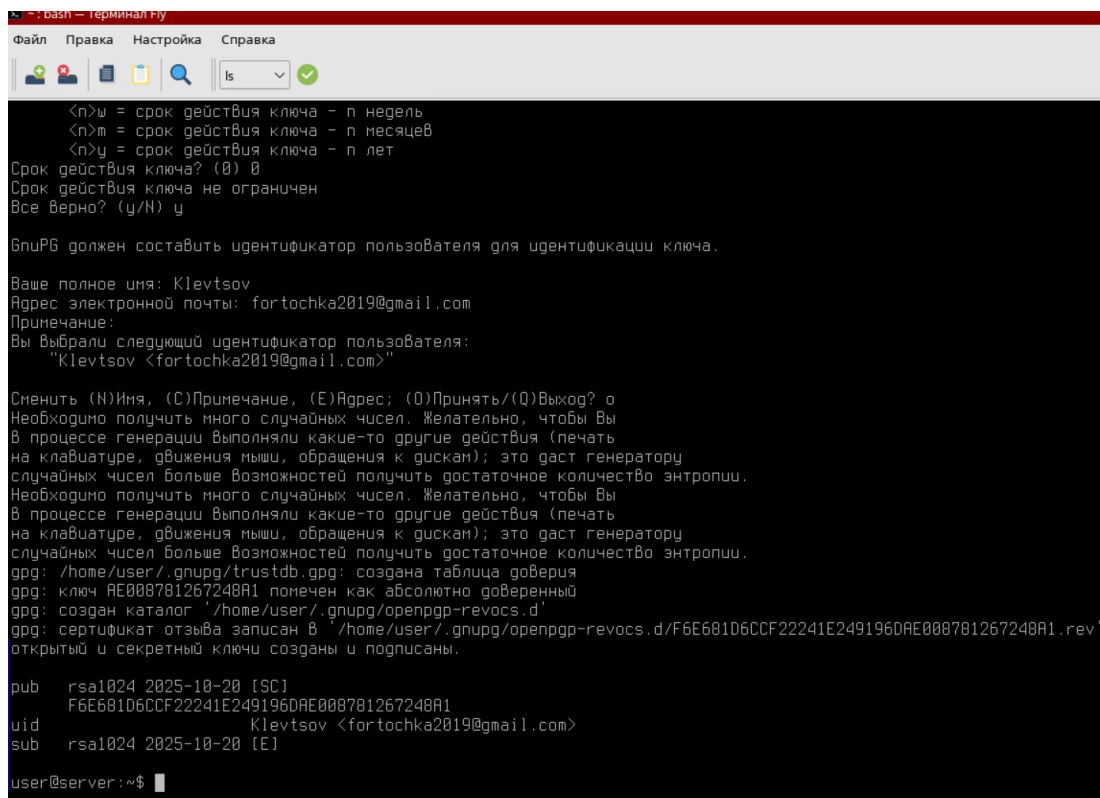
Ход работы:

Работа с криптографическими инструментами

Шифрование файла с использованием GPG

1. Генерация ключей:

Открываем терминал. Вводим команду `gpg --full-generate-key` и следуем инструкциям для создания нового ключевого набора (Рис. 1).



```
bash — терминал Py
Файл  Правка  Настройка  Справка
ls
<n>w = срок действия ключа - n недель
<n>m = срок действия ключа - n месяцев
<n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Klevtsov
Адрес электронной почты: fortotchka2019@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
"Klevtsov <fortotchka2019@gmail.com>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (Q)Принять/(Q)Выход? o
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: /home/user/.gnupg/trustdb.gpg: создана таблица доверия
gpg: ключ AE008781267248A1 помечен как абсолютно доверенный
gpg: создан каталог '/home/user/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/home/user/.gnupg/openpgp-revocs.d/F6E681D6CCF22241E249196DAE008781267248A1.rev'
открытый и секретный ключи созданы и подписаны.

pub  rsa1024 2025-10-20 [SC]
     F6E681D6CCF22241E249196DAE008781267248A1
uid          Klevtsov <fortotchka2019@gmail.com>
sub  rsa1024 2025-10-20 [E]

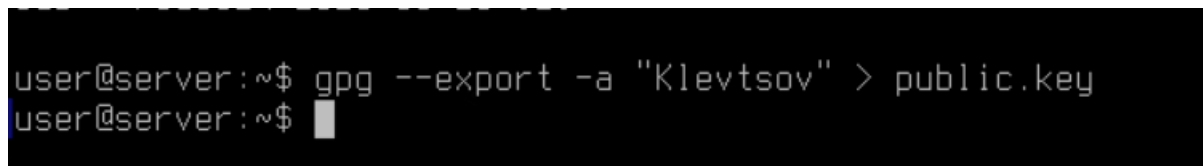
user@server:~$
```

Рис. 1 – Создание нового ключевого набора.

2. Экспорт публичного ключа:

Вводим команду `gpg --export -a "Klevtsov" > public.key`

Опция `-a`, создаёт зашифрованный файл в формате ASCII (Рис. 2).

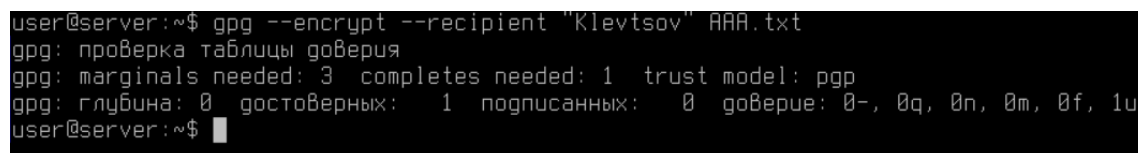


```
user@server:~$ gpg --export -a "Klevtsov" > public.key
user@server:~$
```

Рис. 2 – Экспорт публичного ключа.

3. Шифрование файла:

Вводим команду `gpg --encrypt --recipient "Klevtsov" файл`, где файл - путь к файлу, который нужно зашифровать (Рис. 3).



```
user@server:~$ gpg --encrypt --recipient "Klevtsov" AAA.txt
gpg: проверка таблицы доверия
gpg: marginals needed: 3  completes needed: 1  trust model: pgp
gpg: глубина: 0  достоверных: 1  подписанных: 0  доверие: 0-, 0q, 0n, 0m, 0f, 1u
user@server:~$
```

Рис. 3 – Шифрование файла.

4. Расшифровка файла:

Вводим команду `gpg --output файл --decrypt файл.gpg`, где файл.gpg - зашифрованный файл, а файл - имя файла после расшифровки (Рис. 4).

```

user@server:~$ gpg --output AAA.txt --decrypt AAA.txt.gpg
gpg: зашифровано 1024-битным ключом RSA с идентификатором 097C35B8A6465623, созданным 2025-10-20
"Klevtsov <fortochka2019@gmail.com>"
Файл 'AAA.txt' существует. Запустить поверх? (y/N) N
Введите новое имя файла: RRR
user@server:~$ ls
AAA.txt  AAA.txt.gpg  Desktop  Desktops  inform  public.key  RRR  SystemWallpapers  tmp  windows
user@server:~$ █

```

Рис. 4 – Расшифровка файла.

Работа с контейнерами

1. Запуск Docker

Вводим команду `sudo systemctl start docker` для запуска демона Docker.

Добавляем пользователя в группу `docker` для выполнения команд без `sudo`: `sudo usermod -aG docker $USER` (Рис. 5).

```

user@server:~$ sudo usermod -aG docker $USER
user@server:~$ █

```

Рис. 5 – Добавление пользователя в группу.

2. Запуск контейнера

Запускаем тестовый контейнер: `docker run hello-world` для проверки, что Docker установлен и работает корректно (Рис. 6).

```

user@server:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:6dc565aa63092705211f823c303948cf83670a3903ffa3849f1488ab517f891
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

```

Рис. 6 – Запуск контейнера.

Работа с образами и контейнерами

1. Скачивание образа:

Вводим команду `docker pull ubuntu` для скачивания образа Ubuntu (Рис. 7).

```
user@server:~$ docker pull Ubuntu
invalid reference format: repository name (library/Ubuntu) must be lowercase
user@server:~$ docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
4b3ffd8ccb52: Pull complete
Digest: sha256:66460d557b25769b102175144d538d88219c077c678a49af4afca6fbfc1b5252
Status: Downloaded newer image for ubuntu:latest
docker.io/library/ubuntu:latest
user@server:~$
```

Рис. 7 – Скачивание образа.

2. Запуск контейнеры:

Вводим команду `docker run -it ubuntu` для запуска контейнера с образом Ubuntu и доступом к оболочке (Рис. 8).

Пользователь теперь находится внутри контейнера и может выполнять команды в изолированной среде.

```
user@server:~$ docker run -it ubuntu
root@f9eaff3d5931:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@f9eaff3d5931:/#
```

Рис. 8 – Запущенный контейнер.

Задание на самостоятельное обучение и работу с Docker файлами:

1. Создаём папку и в ней создаём бинарный файл. В файле прописываем параметры для вывода текущих запущенных процессов (`htop`). Создаём контейнер и запускаем его (Рис. 9)

```

user@server:~$ sudo systemctl start docker
[sudo] пароль для user:
user@server:~$ docker pull ubuntu:latest
latest: Pulling from library/ubuntu
Digest: sha256:66460d557b25769b102175144d538d88219c077c678a49af4afca6fbfc1b5252
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest
user@server:~$ mkdir docker-ubuntu-htop && cd docker-ubuntu-htop
user@server:~/docker-ubuntu-htop$ nano Dockerfile
user@server:~/docker-ubuntu-htop$ ls
Dockerfile
user@server:~/docker-ubuntu-htop$ docker build -t my-ubuntu-htop .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/3 : FROM ubuntu:latest
--> 97bed23a3497
Step 2/3 : RUN apt update && apt install -y nano htop && rm -rf /var/lib/apt/lists/*
--> Using cache
--> b416ccbc61f6
Step 3/3 : CMD ["htop"]
--> Using cache
--> 2080b7dacf5b
Successfully built 2080b7dacf5b
Successfully tagged my-ubuntu-htop:latest
user@server:~/docker-ubuntu-htop$ docker run -it --rm my-ubuntu-htop
user@server:~/docker-ubuntu-htop$ █

```

Рис. 9 – выполнение дополнительного задания

2. Демонстрируем, что контейнер действительно создан (Рис. 10).

```

user@server:~/docker-ubuntu-htop$ docker run -it --rm my-ubuntu-htop /bin/bash
root@49cc8bd28781:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@49cc8bd28781:/# █

```

Рис. 10 – Нахождение пользователя в контейнере.

3. Демонстрируем функциональность бинарного файла (Рис. 11).

```

CPU[|||||] 0.7% Tasks: 1, 0 thr, 0 kthr: 1 running
Mem[|||||] 743M/3.82G Load average: 0.50 0.13 0.04
Sup[|||||] 14.9M/975M Uptime: 20 days, 08:06:40

Main 1/0
PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
1 root 20 0 5092 3776 2988 R 0.0 0.1 0:00.03 htop

```

Рис. 11 – Функциональность бинарного файла.

Вывод:

В ходе выполнения лабораторной работы были изучены особенности процесса создания и работы Docker и шифрование файла с использованием GPG. Изучен процесс работы с криптографическими инструментами и инструментами шифрования.